Common Problems

Connect Four

By Evan King · Published Dec 8, 2025 · easy · Asked at:

Watch Video Walkthrough

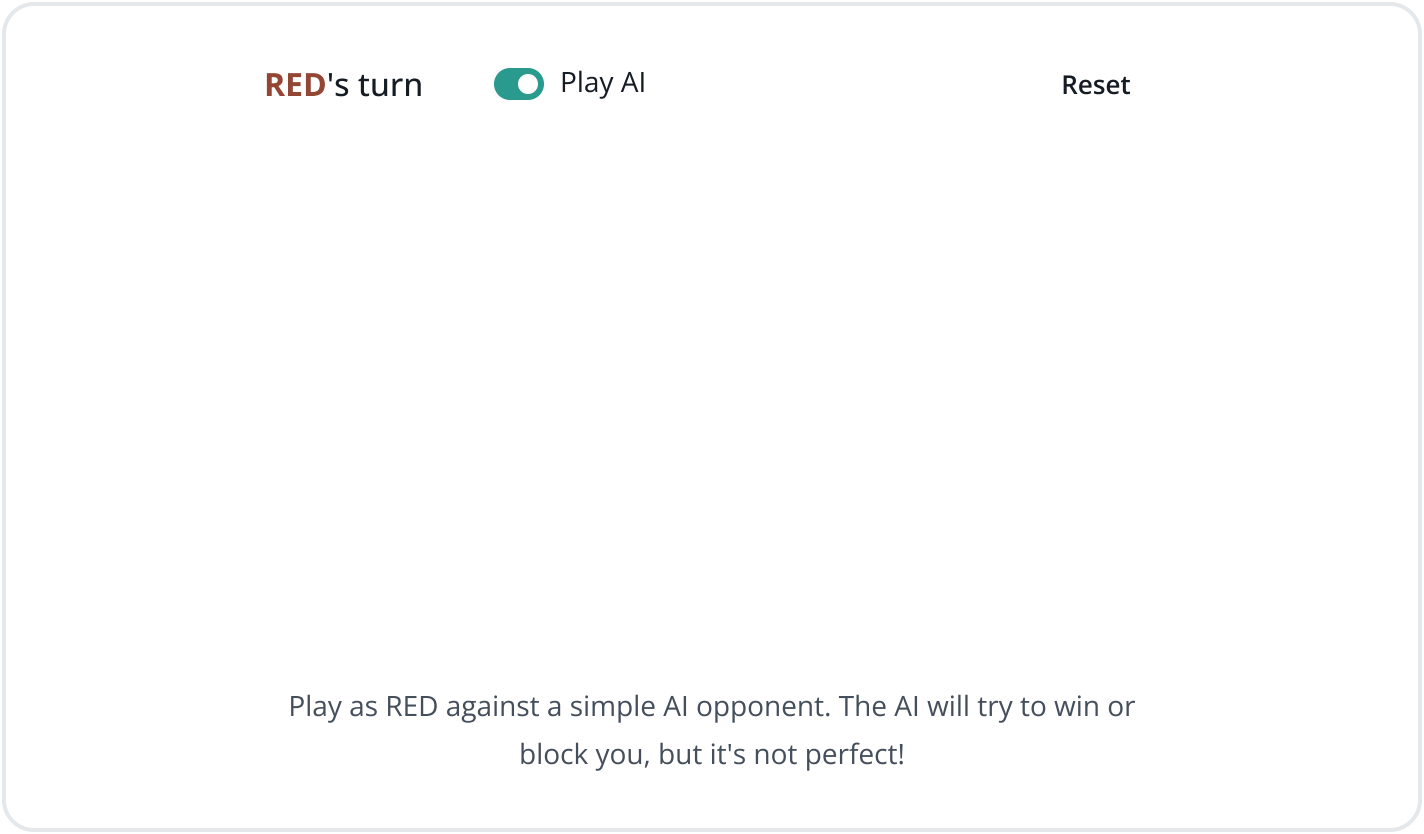
Watch the author walk through the problem step-by-step

Watch Now

Understanding the Problem

🔍 What is Connect Four?

Connect Four is a two-player connection game where the players take turns placing their pieces in a 7x6 grid. The first player to connect four of their pieces in a row, column, or diagonal wins.



Requirements

As the interview begins, we'll likely be greeted by a simple prompt to set the stage for the architecture we need to design.

> "Build the object-oriented design for a two-player Connect Four game. Players take turns dropping discs into a 7-column, 6-row board. The first to align four of their own discs vertically, horizontally, or diagonally wins."

Before jumping into class design, you should ask questions of your interviewer. The goal here is to turn that vague prompt into a concrete specification - something you can actually build against.

Clarifying Questions

> The goal is to surface ambiguity early and get to a concrete spec. This mirrors the same process we use when clarifying requirements on a real project.

A reliable way to structure your questions is to cover four areas: what the core actions are, how errors should be handled, what the boundaries of the system are, and whether we need to plan for future extensions.

Here's how a conversation between you and the interviewer might go:

> **You:** "How do players interact with the game? Do they just specify a column number and the disc drops?"
> **Interviewer:** "Yes, players choose a column from 0 to 6, and the disc falls to the lowest available spot."

Good. You've confirmed the main action. Now think about how the game ends.

> **You:** "What are all the ways a game can end? Is it just four in a row, or are there draws?"
> **Interviewer:** "Four in a row—vertical, horizontal, or diagonal—wins. If the board fills up with no winner, it's a draw."

Now you know the win and draw conditions. Next, think about what happens when things go wrong.

> **You:** "What should happen if someone tries to drop a disc in a column that's already full? Should I return an error, throw an exception, or just ignore it?"
> **Interviewer:** "Return false or raise an error. Don't let invalid moves break the game state."

> **You:** "And what if a player tries to move out of turn?"
> **Interviewer:** "Same thing. Reject it clearly."

Now you're covering error handling. Next, figure out the scope.

> **You:** "Are we designing this to support one game at a time, or do we need to handle multiple concurrent games?"
> **Interviewer:** "Just one game. Keep it simple."

> **You:** "Got it. And is this backend logic only, or do we need UI support as well?"
> **Interviewer:** "Backend only. Someone else will handle rendering."

That last question matters more than you might think. If you need UI support, you'll want methods like `getBoardState()` or `getValidMoves()` so something can render the board. If it's backend-only, you can keep the API minimal and focused purely on game rules.

Finally, check if there are any future features to plan for.

> **You:** "Do we need to track move history or support undo?"
> **Interviewer:** "No, don't overcomplicate it."

> **You:** "What about board size—does it need to be configurable, or always 7x6?"
> **Interviewer:** "Always 7x6."

Perfect. You've now clarified scope and ruled out unnecessary complexity.

Final Requirements

After that back-and-forth, you can write down the final requirements, as well as any learning about what is out of scope, on the whiteboard.

FINAL REQUIREMENTS

```
Requirements:
1. Two players take turns dropping discs into a 7-column, 6-row board
2. A disc falls to the lowest available row in the chosen column
3. The game ends when:
    - A player gets four discs in a row (vertical, horizontal, or diagonal). They
    - The board is full. It's a draw.
4. Invalid moves should be rejected clearly:
    - Dropping in a full column.
    - Moving out of turn.
    - Moving after the game is over.

Out of scope:
- UI support
- Concurrent games
- Move history
- Undo
- Board size configuration
```

Core Entities and Relationships

With a clear set of requirements in hand, the next step is figuring out what objects we need and how they interact.

Start by asking yourself: what are the main "things" in this problem? Look for nouns in your requirements and think about what responsibilities each one should have. In Connect Four, a few jump out immediately: the game itself, the board where pieces are placed, and the players making moves.

> A common mistake is putting everything in one giant class or splitting things unnecessarily. Good design means each class has a single, clear job. The Board manages grid state and placement rules. The Game orchestrates turns and win checking. Players are just data—names and which color they're playing.

For Connect Four, here's what makes sense:

Entity	Responsibility
Game	The orchestrator. Holds the Board, tracks which Player's turn it is, manages game state (in progress, won, draw), and enforces turn-based rules. When a player makes a move, Game validates it, tells Board to place the disc, checks if that move won, and switches turns.
Board	The 7x6 grid where discs live. Owns the grid state and handles disc placement. Knows how to check if a column is full, where a disc should fall, and whether four discs are connected. Doesn't care about whose turn it is or who's winning.
Player	Represents a person in the game. Simple data holder with a name and disc color. No game logic here.

Class Design

Now that we've identified the three core entities, the next step is defining their interfaces. This means deciding what data each class holds and what methods it exposes to the outside world.

I recommend starting with a top-down approach. Begin by designing the Game class first since it's the orchestrator and primary entry point. Once we've defined Game's interface, work your way down to Board and Player. This keeps us focused on the public API instead of getting lost in implementation details.

For each entity, we'll use our requirements to derive both the state and the behavior (methods) of the class. Let's start with Game.

Game

The `Game` class is the orchestration layer. External code should interact with the game only through this class: creating a new game, asking whose turn it is, making moves, and checking whether the game is over.

You can derive almost everything about `Game` directly from the final set of requirements. During the interview, revisit the requirements and ask: "What does the game need to remember to enforce this?". This is how you'll derive the state of the `Game` class.

Requirement	What `Game` must track
"Two players take turns dropping discs into a 7-column, 6-row board"	The two players, whose turn it is, and the board
"The game ends when a player wins or the board is full"	The game state (in progress, won, draw)
"A player gets four discs in a row. They win."	Who won (if anyone)

Let's first weigh some options for the state of the `Game` class.

Bad Solution: Boolean Flags for Game State

Great Solution: GameState Enum

This leaves us with a simple state object:

GAME STATE

```
class Game:
    - board: Board
    - player1: Player
    - player2: Player
    - currentPlayer: Player
    - state: GameState        // IN_PROGRESS, WON, DRAW
    - winner: Player?         // null if no winner yet or draw
```

Importantly, we make `winner` nullable. In a draw, there simply is no winner, which is clearer than overloading a special "NONE" value.

Next, look at the actions the outside world needs to perform. Every method on `Game` should correspond to a concrete need in the problem statement.

Need from requirements	Method on `Game`
"Players take turns dropping discs"	`makeMove(player, column)` - the core action

Need from requirements	Method on `Game`
"Reject moves out of turn"	`getCurrentPlayer()` - caller needs to know whose turn it is
"The game ends when..."	`getGameState()` - caller needs to check if game is over
"A player gets four discs in a row"	`getWinner()` - caller needs to know who won

> While designing, we may discover methods we need that aren't explicitly in your requirements. That's normal. For example, we might realize we need `getCurrentPlayer()` so callers can display whose turn it is. Feel free to go back and add it. This iterative refinement is expected and shows good design thinking.

Adding the methods to the `Game` class, we get:

GAME

```
class Game:
    - board: Board
    - player1: Player
    - player2: Player
    - currentPlayer: Player
    - state: GameState        // IN_PROGRESS, WON, DRAW
    - winner: Player?         // null if no winner yet or draw

    + Game(player1, player2)
    + makeMove(player, column) -> bool
    + getCurrentPlayer() -> Player
    + getGameState() -> GameState
    + getWinner() -> Player?
    + getBoard() -> Board
```

The constructor initializes the game state:

GAME CONSTRUCTOR

```
Game(player1, player2)
    board = Board()
    this.player1 = player1
```

```
    this.player2 = player2
    currentPlayer = player1    // player1 goes first
    state = IN_PROGRESS
    winner = null
```

`makeMove` is the only method that mutates game state. Everything else is read-only. `getCurrentPlayer` lets UI layers show "Player X's turn" without duplicating turn logic, while `getGameState` and `getWinner` let callers check the outcome without digging into internals.

Board

`Board` owns the grid. It knows where discs are, whether a column has space, how discs "fall," and whether a given move creates four in a row.

You can derive its state straight from the requirements:

Requirement	What `Board` must track
"7-column, 6-row board"	Fixed dimensions: number of rows and columns
"A disc falls to the lowest available row in the chosen column"	The current occupancy of each column (the grid)
"The board is full. It's a draw."	Whether there is at least one empty cell left
"A player gets four discs in a row..."	Enough information in the grid to check contiguous discs for a given player

That leads to something like:

BOARD STATE

```
class Board:
    - rows: int            // 6
    - cols: int            // 7
    - grid: DiscColor?[rows][cols]  // null if empty; otherwise the disc color
```

We store `DiscColor` in the grid rather than `Player` to keep the board separately testable. You could store `Player` instead if you prefer—both work as long as you're consistent. But, from a

testability perspective, it's better to store `DiscColor` since it's a simpler type and doesn't require you to mock a `Player` object.

From the outside, `Board` needs to support a small set of actions:

Need from requirements	Method on `Board`
"Check that column has space before placing"	`canPlace(column)`
"A disc falls to the lowest available row"	`placeDisc(column, color)` returns the row
"The board is full. It's a draw."	`isFull()`
"A player gets four discs in a row"	`checkWin(row, column, color)`
UI or external code needs to render the grid	`getCell(row, column)`

That gives you this interface:

BOARD

```
class Board:
    - rows: int = 6
    - cols: int = 7
    - grid: DiscColor?[rows][cols]

    + Board()
    + getRows() -> int
    + getCols() -> int
    + canPlace(column) -> bool
    + placeDisc(column, color) -> int      // returns row where disc lands
    + isFull() -> bool
    + checkWin(row, column, color) -> bool
    + getCell(row, column) -> DiscColor?
```

`Board` encapsulates all the grid math and win detection. `Game` doesn't know how to scan four in a row; it just asks `Board` and updates its own state accordingly.

Player

`Player` represents one participant in the game. From the requirements, the system only needs two things: a way to distinguish the players and a way to know which disc color each one uses.

You can derive the state directly from the problem:

Requirement	What `Player` must track
"Two players take turns..."	A name or ID so the Game can compare players
"their own discs"	The disc color associated with that player

This leads to a minimal class:

PLAYER STATE

```
class Player:
    - name: string
    - color: DiscColor     // RED or YELLOW
```

- `name` lets the Game determine whose turn it is and validate that the caller of `makeMove` matches the expected player. This could be a display name or some stable ID—we just need something Game can compare.
- `color` links placed discs to the correct owner inside the Board grid.

The interface is correspondingly small:

PLAYER

```
class Player:
    - name: string
    - color: DiscColor

    + Player(name, color)
    + getName() -> string
    + getColor() -> DiscColor
```

Player stays deliberately simple. All game flow, move validation, and win logic belong elsewhere.

Final Class Design

With clear class boundaries established, here's how the pieces work together: Game orchestrates the entire flow by validating players, delegating to Board for disc placement, checking win conditions after each move, and switching turns. Board owns the grid state and handles all the physical rules around disc placement and win detection. Player just holds identifying information with no game logic.

FINAL CLASS DESIGN

```
class Game:
    - board: Board
    - player1: Player
    - player2: Player
    - currentPlayer: Player
    - state: GameState        // IN_PROGRESS, WON, DRAW
    - winner: Player?

    + Game(player1, player2)
    + makeMove(player, column) -> bool
    + getCurrentPlayer() -> Player
    + getGameState() -> GameState
    + getWinner() -> Player?
    + getBoard() -> Board

class Board:
    - rows: int = 6
    - cols: int = 7
    - grid: DiscColor?[rows][cols]

    + Board()
    + getRows() -> int
    + getCols() -> int
    + canPlace(column) -> bool
    + placeDisc(column, color) -> int
    + isFull() -> bool
    + checkWin(row, column, color) -> bool
    + getCell(row, column) -> DiscColor?

class Player:
```

```
    - name: string
    - color: DiscColor

    + Player(name, color)
    + getName() -> string
    + getColor() -> DiscColor

enum GameState:
    IN_PROGRESS
    WON
    DRAW

enum DiscColor:
    RED
    YELLOW
```

The design keeps validation and orchestration in Game while physical board rules stay in Board. Player remains a pure data holder with no behavior.

Implementation

With the core classes defined, the next step is walking through how each method actually behaves. Different companies treat this step differently—some want pseudo-code, some want real code, some just want us to talk through it. After you present your class designs, ask your interviewer what level of detail they want.

For each method, follow a consistent pattern:

1. **Define the core logic** - The happy path that fulfills the requirement.
2. **Consider edge cases** - What can go wrong? Invalid inputs, illegal states, boundary conditions.

This structure is natural because you first understand what the method should do, then think about what could break it. Interviewers notice when you systematically identify edge cases—it signals you think about production code, not just the happy path.

If interviewers want code, they'll typically ask for the most interesting methods. In our case, these are:

- `makeMove` to show turn enforcement and game flow

- `placeDisc` to show how discs fall in a column
- `checkWin` to show directional scanning

When writing actual code, aim for clarity over cleverness and try to avoid any premature optimization.

Game

For `Game`, and frankly, the entire design, the core method is `makeMove` it encapsulates the entire game flow and is the most interesting method to implement.

Core logic:

1. Place the disc via `board.placeDisc(column, player.getColor())` -> returns row
2. Check for win via `board.checkWin(row, column, player.getColor())`
3. If no win, check for draw via `board.isFull()`
4. Switch turn if game is still in progress
5. Return true

Edge cases (reject before touching state):

- Game is already over (state is WON or DRAW)
- Wrong player's turn
- Invalid column or column is full (delegated to Board)

We can turn this into a simple pseudo-code implementation:

MAKEMOVE

```
makeMove(player, column)
    if state != IN_PROGRESS
        return false
    if player != currentPlayer
        return false

    row = board.placeDisc(column, player.getColor())
    if row == -1
        return false

    if board.checkWin(row, column, player.getColor())
```

```
            state = WON
            winner = player
        else if board.isFull()
            state = DRAW
        else
            currentPlayer = (player == player1) ? player2 : player1 // switch turn
        return true
```

Notice that we don't check column bounds or whether the column is full in `Game`. That's the Board's responsibility. It knows its own dimensions and state. We let `placeDisc` handle all board-related validation and return `-1` if the move is invalid. This keeps concerns properly separated: Game handles game rules (turns, state), Board handles grid rules (bounds, placement).

We'll want to talk through each decision and weigh your choice against alternatives, weighing the trade-offs as we go. Two common alternatives worth mentioning:

- Have `makeMove` throw exceptions instead of returning false on invalid moves. This can be fine in some languages, but in an interview, a simple boolean result often keeps things clearer. If unsure, ask your interviewer.
- Have `makeMove` implicitly use `currentPlayer` without taking player as an argument. This is simpler if the only code calling makeMove is your own. Taking player explicitly can be useful if we imagine a networked setting where moves arrive tagged with a player. Either choice is reasonable as long as we explain it.

Board

For the `Board` class, the most interesting methods are `placeDisc` and `checkWin` so we'll walk through each of them in detail.

Starting with `placeDisc`, you can derive the core logic and edge cases as follows:

Core logic:

1. Find the lowest empty row in that column—start from `row = rows - 1` and move upward until you find `grid[row][column] == null`
2. Place the disc—set `grid[row][column] = color`
3. Return the row where the disc landed

Returning the row lets `Game` pass `(row, column, color)` into `checkWin` without re-scanning. You can mention an optimization: keep a `heights[cols]` array that tracks the next free row per column. For a 7x6 board, the simple loop is fine.

Edge cases:

- Column index out of bounds -> return error or -1
- Column is full -> return error or -1

We keep all validation inside `placeDisc` rather than having `Game` call `canPlace()` first. This way, the Board owns all grid-related validation in one place, and `Game` simply checks if the return value is `-1` to know the move failed. This is cleaner separation of concerns.

In pseudo-code, this looks like:

PLACEDISC

```
placeDisc(column, color)
    if column < 0 || column >= cols
        return -1
    if !canPlace(column)
        return -1

    for row = rows - 1 down to 0
        if grid[row][column] == null
            grid[row][column] = color
            return row
    return -1
```

Moving onto the `checkWin` method, you can derive the core logic and edge cases as follows:

Core logic:

1. Define the four directions: horizontal `(0, 1)`, vertical `(1, 0)`, diagonal down-right `(1, 1)`, diagonal up-right `(-1, 1)`
2. For each direction, count contiguous discs in both directions from `(row, column)`
3. If any direction reaches 4 or more, return `true`

Edge cases:

- Row or column out of bounds → return false
- Cell at (row, column) doesn't match the given color → return false

This is a common place where candidates over-engineer the solution. Let's break down why by weighing the trade-offs of two possible approaches.

Bad Solution: Separate Win Checker Classes

Great Solution: Unified Directional Vector Approach

If we go with the simple option (as I suggest you do) then we can add our edge cases and end up with something like this:

CHECKWIN

```
checkWin(row, col, color)
    if row < 0 || row >= rows || col < 0 || col >= cols
        return false
    if grid[row][col] != color
        return false

    directions = [[0,1], [1,0], [1,1], [-1,1]]
    for dr, dc in directions:
        count = 1
        count += countInDirection(row, col, dr, dc, color) # move in the directior
        count += countInDirection(row, col, -dr, -dc, color) # move in the opposit
        if count >= 4
            return true
    return false

countInDirection(row, col, dr, dc, color)
    count = 0
    r = row + dr
    c = col + dc
    while inBounds(r, c) && grid[r][c] == color
        count++
        r += dr
```

```
            c += dc
        return count
```

As for the remaining helper methods, they are straightforward but worth showing for completeness:

CANPLACE

```
canPlace(column)
    if column < 0 || column >= cols
        return false
    return grid[0][column] == null    // top row empty means column has space

isFull()
    for c = 0 to cols - 1
        if canPlace(c)
            return false
    return true

inBounds(row, col)
    return row >= 0 && row < rows && col >= 0 && col < cols
```

Player

`Player` has no interesting implementation—just getters for `name` and `color`. Skip this unless the interviewer explicitly asks for it.

Complete Code Implementation

While most companies only require pseudocode during interviews, some do ask for full implementations of at least a subset of the classes or methods. Below is a complete working implementation in common languages for reference.

Game.py | Board.py | Player.py | Python

```
from enum import Enum
from typing import Optional
```

```
class GameState(Enum):
    IN_PROGRESS = "IN_PROGRESS"
    WON = "WON"
    DRAW = "DRAW"


class Game:
    def __init__(self, player1, player2):
        self.board = Board()
        self.player1 = player1
        self.player2 = player2
        self.current_player = player1
        self.state = GameState.IN_PROGRESS
        self.winner: Optional["Player"] = None

    def make_move(self, player, column: int) -> bool:
        if self.state is not GameState.IN_PROGRESS:
            return False
        if player is not self.current_player:
            return False

        row = self.board.place_disc(column, player.color)
        if row == -1:
```

Verification

Let's trace through a quick game to verify the win detection and state transitions work. We start with a partially filled board to keep this concise.

GAME TRACE

```
Initial state:
Row 5 (bottom): [RED, YELLOW, RED, _, _, _, _]
Row 4:          [RED, YELLOW, _, _, _, _, _]
currentPlayer = player1, state = IN_PROGRESS

Move 1: player1 → column 0
  placeDisc(0, RED) → row 3
  checkWin(3, 0, RED)?
    Check vertical: (4,0)=RED, (5,0)=RED → count = 3
    No win yet
  currentPlayer = player2

Move 2: player2 → column 1
```

```
  placeDisc(1, YELLOW) → row 3
  checkWin(3, 1, YELLOW)?
    Check vertical: (4,1)=YELLOW, (5,1)=YELLOW → count = 3
    No win yet
  currentPlayer = player1

Move 3: player1 → column 2
  placeDisc(2, RED) → row 4
  checkWin(4, 2, RED)?
    Check horizontal: (4,1)=YELLOW → no consecutive 4
    No win yet
  currentPlayer = player2

Move 4: player2 → column 3
  placeDisc(3, YELLOW) → row 5
  checkWin(5, 3, YELLOW)? No
  currentPlayer = player1

Move 5: player1 → column 0
  placeDisc(0, RED) → row 2
  checkWin(2, 0, RED)?
    Check vertical down: (3,0)=RED, (4,0)=RED, (5,0)=RED
    count = 1 + 3 = 4 ✓
    Returns true!
  state = WON, winner = player1

Move 6: player2 tries column 1
  state != IN_PROGRESS → returns false immediately
```

This verifies disc placement, vertical win detection, state transitions, and move rejection after game ends.

> It's important to verify, but you don't need to write out all the states like this; it might take too much time. Check with your interviewer, but just verbally going over some test cases is usually enough. Remember, the goal is to catch logical errors before your interviewer finds them.

Extensibility

If time allows, interviewers will sometimes add small twists to test whether your design can evolve cleanly. You typically won't need to fully implement these changes—just explain how your classes would adapt. The depth and quantity of the extensibility follow-ups correlate with the candidate's target level (e.g., junior, mid-level, senior). Junior candidates often won't get any, mid-level may get one or two, and senior candidates may be asked to go into more depth.

> If you're a junior engineer, feel free to skip this section and stop reading here! Only carry on if you're curious about the more advanced concepts.

Below are the most common ones for Connect Four, with more detail than you'd need in an actual interview.

1. "How would you support different board sizes?"

Right now the requirement says the board is always 7x6, so you hardcode that into `Board`. If an interviewer asks about configurable dimensions, the goal is to show that your design already has a natural place to plug this in.

> "Today I fix the board at 6 rows by 7 columns because that's the requirement. If we wanted configurable sizes, I'd make `rows` and `cols` constructor parameters on `Board`. All of the placement and win logic already works for arbitrary dimensions because it relies on `rows`, `cols`, and `inBounds`. `Game` doesn't need to change much: it just chooses what size board to construct."

2. "How would you add undo or move history?"

Undo is a very common follow-up question because it tests whether your design has a clean separation between *orchestration* (Game) and *state* (Board). Since all moves flow through `Game.makeMove`, you already have a single choke point where moves can be recorded.

> “Undo belongs in `Game` because `Game` controls the lifecycle, turn order, and when state changes. I’d keep a `moveHistory` stack. Each time a move succeeds, I push a small `Move` record containing the player, row, and column. Undo would pop the last move, clear that cell in the Board, revert `currentPlayer`, and recalculate game state if needed. The Board doesn’t need any new logic besides maybe an internal `clearCell` method.”

In most interviews that would be enough. However, if the interviewer asks for more detail with regards to the implementation, you can use the following light pseudo-code to guide your answer:

Define a tiny value object:

MOVE

```
class Move:
    - player: Player
    - row: int
    - col: int

    + Move(player, row, col)
```

Add a history stack in `Game`:

GAME

```
class Game:
    - moveHistory: Stack<Move>
```

And then `makeMove` would look like this, using the `Move` value object to store the move history:

MAKEMOVE

```
makeMove(player, column)
    ...
    row = board.placeDisc(column, player.getColor())
    moveHistory.push(Move(player, row, column))
    ...
```

Add a `clearCell` helper to `Board`:

BOARD

```
class Board:
    + clearCell(row, col)
        grid[row][col] = null
```

Then undo becomes:

UNDOLASTMOVE

```
undoLastMove()
    if moveHistory.isEmpty()
        return false

    last = moveHistory.pop()

    // revert board state
    board.clearCell(last.row, last.col)

    // revert turn order
    currentPlayer = last.player

    // recompute state (simplest version)
    state = IN_PROGRESS
    winner = null

    return true
```

You can mention that a production version might recompute win state more cleverly, but for an interview, this is more than enough.

3. "How would you add a computer opponent?"

This follow-up is testing whether you can extend behavior without ripping through all your existing classes. The key is that **rules don't change**: `Game` still enforces turns and validity, and `Board` still owns grid logic. A bot just chooses a column instead of a human.

> "I'd keep the game rules exactly where they are. `Game` and `Board` don't need to change. I'd introduce a small bot component that looks at the current board and returns a column. From `Game`'s perspective, a bot move is just another call to `makeMove(currentPlayer, column)`."

A simple way to describe it is with a separate `BotEngine`:

BOTENGINE

```
class BotEngine:
    + chooseMove(game) -> int
```

A trivial implementation might just pick the first valid column:

CHOOSEMOVE

```
chooseMove(game)
    board = game.getBoard()
    for col = 0 to board.getCols() - 1
        if board.canPlace(col)
            return col
    return -1   // no moves available
```

Then wherever you drive the game loop:

GAMELOOP

```
game = Game(humanPlayer, botPlayer)
bot = BotEngine()

while game.getGameState() == IN_PROGRESS
    current = game.getCurrentPlayer()

    if current == humanPlayer
        column = /* read from UI / input */
    else
        column = bot.chooseMove(game)

    game.makeMove(current, column)
```

The important interview point:

- We **don't** change `Board` at all.
- We **don't** change `makeMove` or the game rules.
- We just add a thin decision-making layer that chooses a column on behalf of a `Player`.

> One alternative which leans heavier into an object-oriented approach is to make `Player` an interface with `HumanPlayer` and `BotPlayer` implementations, where `BotPlayer` uses a `BotEngine` internally to pick a column. Either way, the core design doesn't change—the bot is just a different way to decide which column to pass into `makeMove`.
>
> But I'd argue the `BotEngine` approach is actually the better design. A human player doesn't "do" anything—they're just data. Making `Player` an interface adds abstraction without value. Keeping `Player` as simple data and separating identity from decision making is cleaner.

What is Expected at Each Level?

Ok so what am I looking for at each level?

Junior

At the junior level, I'm checking whether you can decompose the problem into logical pieces and implement a working game. You should identify that you need something to represent the board, something to represent players, and something to orchestrate turns. The exact class names don't matter as much as having sensible responsibilities assigned to each. Your `placeDisc` logic should work - find the lowest empty row and place the disc. Win checking is the tricky part. I expect you to at least check horizontal and vertical wins correctly. Diagonal checking is harder, and it's fine if you need hints. Edge cases like full columns or playing out of turn should be handled, even if your error handling is basic (returning false is fine). If you can play a complete game from start to finish with your code and it correctly identifies a winner or draw, you're doing well.

Mid-level

For mid-level candidates, I expect a cleaner separation of concerns without needing guidance. Game should handle orchestration and turn management. Board should own grid state and win detection. Player should be minimal data, not loaded with game logic. Your `makeMove` method should validate state before mutating anything—check game state, check turn order, check column validity, then place. The win-checking implementation should handle all four

directions cleanly. I like seeing the directional vector approach (`(dr, dc)` pairs) rather than four separate methods, because it shows you recognize the pattern. You should be able to discuss at least one extensibility scenario, like undo or configurable board size, and explain where the changes would live without actually implementing them.

Senior

Senior candidates should produce a design that I'd be comfortable reviewing as production code. The class boundaries should be obvious and well-justified. You should proactively point out design decisions: why Player is just data, why GameState is an enum rather than boolean flags, why win checking lives on Board rather than Game. Your `checkWin` implementation should be elegant - the direction vector pattern with a single `countInDirection` helper that handles all four cases. I expect you to catch your own edge cases during implementation, not wait for me to point them out. If I ask about extensibility, you should be able to discuss multiple approaches with tradeoffs. For adding a bot opponent, you'd recognize that game rules don't change, you just need a decision making component that picks columns. Strong senior candidates often finish early and can discuss how the design would change for a networked multiplayer version or how you'd add spectator support.